Report

Retrieval-Augmented LLM System

Name: Alexander Davidson

Generative AI Use Declaration

System models: ‘text-embedding-3-small’ model for creating the query and document embeddings for dense ranking. ‘gpt-5-nano’ model for judging options for each document.

AI use for language and formatting: I used Google Gemini only for Condition 2 surface polish of sections 1, 2 and 3 in WRITEUP.pdf. I used it for clarity and language polishing. I reviewed and revised the edits after.

1. Best System Overview

Summarise what Best produces and the methods retained in it.
Best produces a probability vector over the question's exact option strings, representing the judged option mix of the evidence. Exactly 100 unique document IDs, ordered judged-relevant first (dense order within each tier) and judged non_relevant last. For retrieval it used dense cosine and for judging it used zero-shot LLM judging with B0 token-overlap retained as a last-resort fallback.

2. Implemented Methods and Choices

2.1 Data access

Best used the document embeddings, where one embedding call encodes the query, so the vector is anchored to both the topic and the answer vocabulary. This dense ranking done a direct cosine scan, so the inverted index built from Whoosh is not queried. The retrieval produces an exact, exhaustive top 100 cosine ranking. Dense (best) beat RRF on Kaggle which is why it was rejected. RRF searched the inverted index each query for a BM25 ranking, running in parallel with the dense scan. It ranked each system independently to a pool of 1000, then merged by RRF. RRF deliberately used only ranks, not score magnitudes, so cosine similarities and BM25 scores did not need normalisation to be comparable. The BM25's contribution consisted largely of documents dense had correctly ranked low and because RRF weights every rank position equally, a BM25 rank-1 hit on a lexically-matching but non-gold post counts as much as dense's semantically justified rank-1 hit. This promoted BM25-only non-relevant posts and displacing dense's well-ordered relevant ones. This extra index dependency and the BM25 latency bought a net loss on cost, with no increase in evidence ranking scores, so RRF was rejected.

2.2 Data analysis

Best used a fixed zero-shot system prompt which states the decision rule, the non_relevant case, an exact JSON output schema, reasons capped at eight words, and a 'never guess' instruction. The user message was purely structural with no demonstrations, no reasoning steps, and no rephrasing. Replies first flowed through a lenient JSON extraction with repair, option matching, and a retry ladder so exactly judge_k valid judgements always exist. Aggregation counts one vote per non-non_relevant judgement and normalises. Judging was treated as a sampling process, so a zero-shot resample established the noise floor first, so every claim was judged against measured variance rather than raw changes which is why few-shot was rejected. Few-shot used a config gate 3 judge shots and is prepended to the user prompt for a real mini support labelled Console, a synthetic cross-question Yes example, and a clearly irrelevant document labelled non_relevant. The scoring target was in each question's own option mix, but question, biased label frequencies toward demonstrated options, and caused distribution shift, so few-shot was rejected.

3. Comparison Evidence

Use the Runs Table to summarise B0, Best and the three controlled comparisons. B0 shows the provided development scores. The other rows contain illustrative examples. Replace them with your own runs.

Run	Data access	Data analysis	Dev Dist.	Dev MAP	Per-run API use (tokens / USD)
B0	Boolean OR, document-ID order	Token-overlap judge	0.1633	0.0003	0
Best	Dense, dense_pool=1000	LLM JSON judge, top 100	0.74192	0.47315	22,265 / $0.00307
Access-Var	Reciprocal Rank Fusion	Same as Best	0.68888	0.39733	253,374 / $0.03227

Run	Data access	Data analysis	Dev Dist.	Dev MAP	Per-run API use (tokens / USD)
Analysis-Var	Same as Best	Few-shot	0.65573	0.46763	346,944 / $0.03975
Config-Var	dense_pool=100	Same as Best	0.72659	0.45655	7,489 / $0.00086

Access-Var adds a second, independent access path (Whoosh BM25 queries) to the dense scan. It is the least accurate of the LLM-judged runs, this meant more access paths did mean more information. Config-Var costs little in compute and API spend but gives up measurable quality (0.72659 / 0.45655). The saving is negligible (under a cent) while the quality loss is roughly the size of the entire measured floor noise (±0.016). Best with the zero-shot LLM judge spends only 22,256 tokens / $0.00307 and obtains both best scores. Most of that cost is amortised over the response cache. Best's prompts are the canonical ones, so repeated runs reuse replies. The clearest failure is Access-Var as it assumed that the lexical and semantic access were complementary, yet it spent 11 times Best's API budget and lost on both metrics. Best wins because it is optimal for this problem, not just because of the scores. Dense + zero-shot LLM judge with relevance ordered supports takes both metrics (0.74192 / 0.47315) at roughly 1/10th to 1/15th the API cost of either variant. The more expensive methods demonstrated, in measured runs that additional paths or prompt tokens can actively hurt both what the metrics and the budget.

3.1 Method comparisons

Only the retrieval method's data access was changed from dense cosine to RRF, with Whoosh BM25 (k1 = 1.2, b = 0.75) and the dense cosine scanning. The best zero-shot LLM judge stayed the same over the judged top 100, aggregation, relevance-first support ordering, validation and the submission format stayed the same. The two access paths overlapped only 29/100 for their top-100 lists, but on the full track the fused ranking shared just 8.8/20 documents with dense's judged top-20 on average: fusion replaced dense's documents with BM25's lexical matches. The consequence is visible in both scores MAP fell to 0.39733 because non-gold posts were promoted above correctly ranked gold supports, influencing AP. Distribution fell to 0.6888 because the judge analysed a substantially different document set and therefore cast a different vote mix. This method was rejected as it was worse on both metrics at more than ten times the cost. The experiment was still a useful negative result where list complementary does not imply usable signal when one list's marginal documents are mostly irrelevant.
Only the analysis prompt was changed with 3 judge shots prepended three hand-verified question, options, doc -> judgement) demonstrations. A real support labelled Console, a synthetic cross-question Yes, and an irrelevant document labelled non_relevant to the user message. Best's dense data access method stayed the same, as well as the system prompt, parser, retries, aggregation, ad support ordering were untouched. The examples altered what the judge voted, not what it saw, this led to distribution falling to 0.65573. Two of three demonstrations showed a Mobile/Console-heavy option mix from a different question, pulling per-question vote frequencies toward the demonstrated labels rather than each question's own mix. With retrieval fixed, only the relevance labels feeding support ordering could touch MAP, and they shifted only marginally (0.46763). Cost went up, where demonstrations inflated every batche's input and invalidated all cache keys, so every call was fresh (346,944 tokens / $0.03975). This method was rejected, as it spent the largest budget but still documented as an advanced technique.

3.2 Configuration comparison

The setting that changed was the dense pool from 1000 (best) to 100. Dense pool controlled how any candidates were retained during the dense scan. dense_rank walked the 183k-document embedding memmap in batches of 8192 and, after each batch, merged into a running candidate set capped at the pool which shrunk the working set during traversal to exactly the size of the final output of 100 supports. Everything else stayed the same as above, the dense method with its cosine scoring, query text, zero-

shot LLM judge etc. API used collapsed to 7,489 tokens / $0.00086. This meant that retrieval barely changed and the judged documents still matched the response cache almost completely. If pool=100 had re-ranked the corpus, we would have seen something like RRF's 253,374 fresh token count instead. With the pool pinned at exactly 100, some documents in or out of the judged set changed a few tie-order positions in the support list. Distribution fell to 0.72659 and MAP to 0.45655. These scores just below the noise floor i.e. in the range where judge resampling alone can produce movement. So, this run demonstrated little to no ordering benefit either. At best Config-Var is neutral, at worst marginally worse. The trade-off the dense pool settings controls compute vs quality. The pool of 100 tightens the candidate set to just the output size. So, any future method needing >100 documents becomes more sensitive to tie-order effects.

4. Case Study

Development question: 'How would you describe your household's financial situation?'
Predicted distribution: Option: 'Live comfortably'; 0.4490. Option: 'Meet my basic expenses with a little left over for extras'; 0.2755. Option: 'Just meet my basic expenses'; 0.2551. Option: 'Don't even have enough to meet basic expenses'; 0.0204.
For the option 'Live comfortably'. Ranked evidence puts comfort statements at the top ranks 1, 4, 7, 9, 11 and 12 were all judged 'Live comfortably'. T5MQR8BC9R (rank 4) is unambiguous: 'Our household income is $170K. We live in a MCOL area. I finally feel financially secure.' This is in explicit comfort claim, directly supporting the probability. Evidence that contradicted a predicted share was 6N4C3XJJ6 (rank 1) was also voted 'Live comfortably' yet its own text and the judge's reason ('text shows struggle and little left over') say otherwise: 'I feel constantly like I don't have enough at the end of the month… I'd be hard-pressed…'. This vote most plausibly belongs to 'Just meet my basic expenses', meaning the predicted 0.449 is inflated by at least this one misplaced vote. One limitation is the single-shot judging of borderline posts. Each document got exactly one label from one batch, with no consistency check. Ambiguous posts like those answering, 'it depends,' or mixing signals like rank 1 are resolved by whichever way the model leans that call. An improvement could be majority voting per document with 3 judge samples. Judge each document three times in differently composed batches and keep the majority label. The rank-1 label above would require 2-of-3 agreement to survive. A model that reads 'struggle to save' as 'Live comfortably' once is unlikely to do so twice more, so borderline documents settle on their most defensible option while clear-cut ones are unchanged. Cost is the trade-off, however with ~3 times judging calls, lifting Best's per-run spend from $0.00307 to roughly $0.01.

Evidence Summary Table

Purpose. Tell the marker where each method claim and reported result can be verified. This table is required for the Report Quality mark. If it is missing, Report Quality receives zero. It is excluded from the page limit.

What to list. Locate each claimed method by ZIP-relative code path and function or class name. Methods may share a row and function, for example LLM-based query rewriting and RRF within retrieve_and_rank. Identify the code for each. For Kaggle runs, include the config and run folder with JSONL, CSV, and screenshots named kaggle_distribution.png and kaggle_evidence.png. Leave the run-evidence cell blank for methods without reported results. Add rows as needed.

Distribution and Evidence tracks may have different final submissions. Complete the additional row only for a final run not already listed in this table.

Report section	Run or method	Code location	Config and evidence files
3	B0	retrieval/baseline.py b0_retrieve judging/baseline.py b0_judge	Config: full_b0_config.ini Provided B0 scores Full run and Kaggle evidence optional

Report section	Run or method	Code location	Config and evidence files
2.1, 2.2 and 3	Best	retrieval/rankers.py retrieve_and_rank judging/judges.py judge_evidence	Config: full_config.ini Run folder: runs/best/ Screenshots: kaggle_distribution.png kaggle_evidence.png
3.1	Access-Var	retrieval/rankers.py retrieve_and_rank	Config: access_var.ini Run folder: runs/access_var/ Screenshots: kaggle_distribution.png kaggle_evidence.png
3.1	Analysis-Var	judging/judges.py judge_evidence	Config: analysis_var.ini Run folder: runs/analysis_var/ Screenshots: kaggle_distribution.png kaggle_evidence.png
3.2	Config-Var	retrieval/rankers.py retrieve_and_rank	Config: config_var.ini Run folder: runs/config_var/ Screenshots: kaggle_distribution.png kaggle_evidence.png
4	Case study Best run	retrieval/rankers.py retrieve_and_rank judging/judges.py judge_evidence	Config: full_config.ini Run folder: runs/best/ Files: retrieved.jsonl, judgments.jsonl, submission.csv Screenshots: kaggle_distribution.png kaggle_evidence.png